Assembly to C++ Guidelines and Rubric

CS 410 Assembly to C++ Guidelines and Rubric

Overview

In previous activities, you have converted C++ code into assembly code. In this activity, you will reverse the process and demonstrate your ability to convert assembly code into C++ code. You will complete and submit the Assembly to C++ Activity Template Word Document.

Prompt

For this activity, you will analyze three examples of assembly code. For each example of assembly code, you must address the following rubric criteria:

1. Convert the assembly code into C++ code.
    - Use the Assembly to C++ Activity Template to complete this step.
2. Explain the functionality of the converted C++ code.
    - Use the Assembly to C++ Activity Template to complete this step.
    - Consider which blocks of assembly code are skeleton code versus actual parts from the C++ program.

What to Submit

Assembly to C++ Template

This should be a Word Document. Use the Assembly to C++ Activity Template to convert the three examples of assembly code and explain the function of the converted assembly to C++ code.

AI Usage

If you use gen AI tools to support your work on this assignment, be sure to follow these AI usage guidelines. You must acknowledge your use of these tools in your work. Guidelines on how to cite AI tools can be found in this Shapiro Library guide.

Assembly to C++ Rubric

Criteria	Exceeds Expectations (100%)	Meets Expectations (85%)	Partially Meets Expectations (55%)	Does Not Meet Expectations (0%)	Value
Assembly to C++ Conversion	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner	Converts each block of assembly code into C++ code with minimal errors	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include converting each block of assembly code into C++ code with fewer errors	Does not attempt criterion	45
Explanation of Functionality	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner	Explains the functionality of the C++ code with minimal errors and adequate detail	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include explaining the functionality of the C++ code with	Does not attempt criterion	45

			minimal errors and richer detail		
Clear Communication	Exceeds expectations with an intentional use of language that promotes a thorough understanding	Consistently and effectively communicates in an organized way to a specific audience	Shows progress toward meeting expectations, but communication is inconsistent or ineffective in a way that negatively impacts understanding	Shows no evidence of consistent, effective, or organized communication	10
				Total:	100%

Activity Details

- Task: View this topic